

Phase Vocoder Voice Changer

Independent speed & pitch control for speech, with transient preservation

CSE220 — Signals and Systems

The Problem

Speed and pitch are physically glued together

On a computer, playing a signal faster or slower is the same operation as changing its sample rate — so pitch moves with it, whether you want it to or not.

- Speed up naively → pitch goes **up** (chipmunk effect).
- Slow down naively → pitch goes **down** (drone/slow-motion effect).
- Almost nobody actually wants that coupling — they want *one* knob, not both moving together.

That coupling is exactly what this project removes.

What Our Software Does

Speed

$0.25\times - 2.4\times$
independent of pitch

Pitch

$\pm 3-12$ semitones
independent of speed

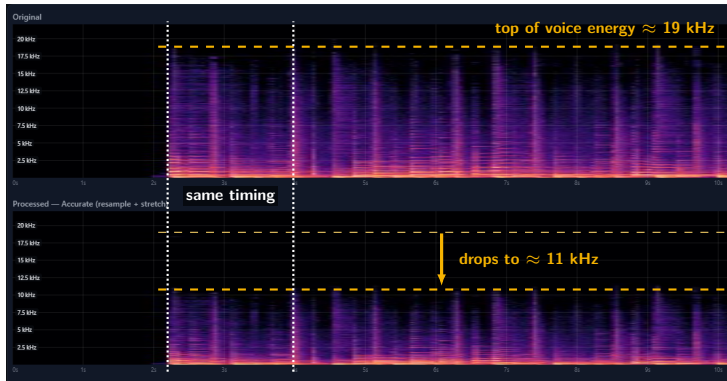
Transients

consonants stay
sharp, not smeared

- Input: any file `soundfile`/FFmpeg can decode (WAV, MP3, FLAC, OGG), or a live mic recording.
- Two quality/cost modes: **Fast** (cheap, small pitch shifts) and **Accurate** (resample + stretch, wider range).
- A built-in **naive-resample** comparison mode, to demonstrate the problem this solves.

Spotting the Change — How to Read the Result

Pitch shift down: -9 semitones, speed $1.0\times$ (Accurate mode)



How to read a spectrogram

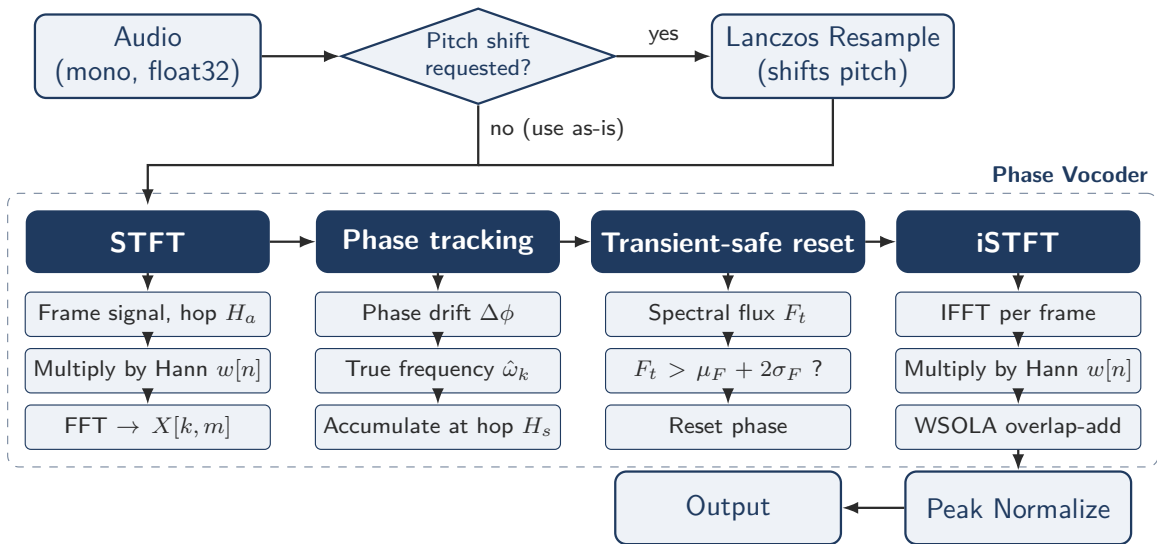
- x -axis = time, y -axis = frequency, brightness = energy.
- Horizontal bands = harmonics (pitch of vowels).
- Vertical streaks = transients (consonant onsets).

What changed

- Top = original, bottom = our output.
- Every band moves **down** by the same ratio: $2^{-9/12} \approx 0.59$, so $19 \text{ kHz} \times 0.59 \approx 11 \text{ kHz}$.
- Onsets (dotted lines) stay at the **same seconds** — the clip length is unchanged.

Pitch moved, time did not.

Solution Overview: The Pipeline



Stage 1: Audio Loading

Backup — for Q&A

Input: any decodable audio file or mic recording.

Output: 1-D mono float32, range $[-1, 1]$, native sample rate.

Algorithm: read via `soundfile`; down-mix multi-channel by averaging.

$$x_{\text{mono}}[n] = \frac{1}{C} \sum_{c=1}^C x_c[n]$$

Why: every later stage assumes one real 1-D signal.

Viva note: stereo width is permanently lost here (known limitation).

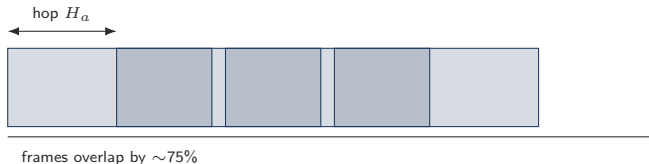
Stage 2: STFT

Backup — for Q&A

Input: mono waveform. **Output:** complex matrix $X[k, m]$ (freq. bin $\times$ frame).

Steps: (1) cut overlapping frames, hop H_a (2) multiply each frame by the Hann window
(3) FFT each frame.

$$X[k, m] = \sum_{n=0}^{N-1} x[mR + n] w[n] e^{-j \frac{2\pi}{N} kn}$$



Stage 3: Phase Vocoder

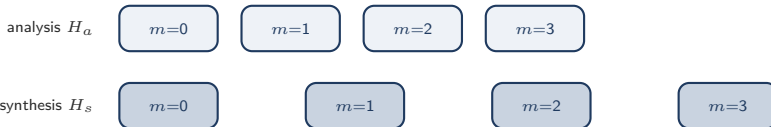
Backup — for Q&A

Input: STFT matrix, target speed factor, optional pitch factor.

Output: phase-corrected spectrum, re-hopped at synthesis spacing H_s .

Algorithm: estimate true instantaneous frequency per bin from phase drift, re-accumulate phase at H_s .

$$\hat{\omega}_k = \omega_k + \frac{\Delta\phi - \omega_k H_a}{H_a}, \quad \phi_{\text{out}}[m] = \phi_{\text{out}}[m-1] + \hat{\omega}_k H_s$$



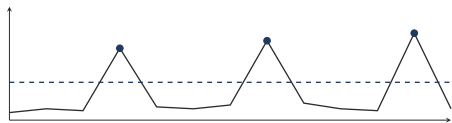
Stage 4: Transient Detection & Phase Reset

Backup — for Q&A

Input: STFT magnitude $|X[k, m]|$. **Output:** per-frame transient flag; phase reset instead of accumulated on that frame.

Algorithm: spectral flux vs. an adaptive threshold from recent history.

$$F_t = \sum_k \max(0, |X_t(k)| - |X_{t-1}(k)|), \quad \text{transient if } F_t > \mu_F + 2\sigma_F$$



(schematic) dashed line = adaptive threshold $\mu_F + 2\sigma_F$; dots = phase-reset frames.

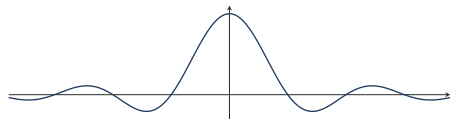
Stage 5: Lanczos Resampling

Backup — for Q&A

Input: audio, stretch factor s . **Output:** signal of length $N \cdot s$.

Algorithm: windowed-sinc interpolation; cutoff lowers to new Nyquist when compressing ($s < 1$).

$$r = \min(1, s), \quad L_a(x) = r \cdot \text{sinc}(rx) \cdot \text{sinc}(rx/a)$$



$a = 8$ taps by default — same operation used for the naive-resample comparison clip, minus the phase-vocoder correction step.

Stage 6: Final Output

Backup — for Q&A

Input: reconstructed audio from overlap-add. **Output:** peak-normalized float32 audio; Accurate mode also writes a WAV.

Steps (iSTFT): (1) IFFT each frame (2) multiply by the Hann window (3) WSOLA overlap-add at hop H_s , then scale by loudest sample.

$$y[n] = \frac{\sum_m y_m[n - mR] w[n - mR]}{\sum_m w^2[n - mR]}, \quad x_{\text{norm}} = \frac{x}{\max |x|} \times 0.99$$

Viva note: at identity settings (speed 1.0, shift 0) the app reports a round-trip SNR — a sanity check, not a general quality metric.

Trade-offs: Window Size, Overlap & Limits



- Overlap rises with $|\text{effective stretch} - 1|$: **75%** default $\rightarrow$ up to **90%** for extreme stretch.
- Hard limit (code-enforced): compound time-stretch **0.25 $\times$ –3.0 $\times$** .
- Pitch shift: ± 3 st (Fast mode) / ± 12 st (Accurate mode); artifacts grow past $\sim \pm 9$.
- Playback speed (UI): 0.25 $\times$ –2.4 $\times$.

Transient Preservation

Problem: accumulating phase through a consonant onset smears it into neighboring frames.

Fix: detect the onset via spectral flux, then **reset** synthesis phase to the analysis phase at that frame — resume normal accumulation right after.

$$F_t = \sum_k \max(0, |X_t(k)| - |X_{t-1}(k)|)$$

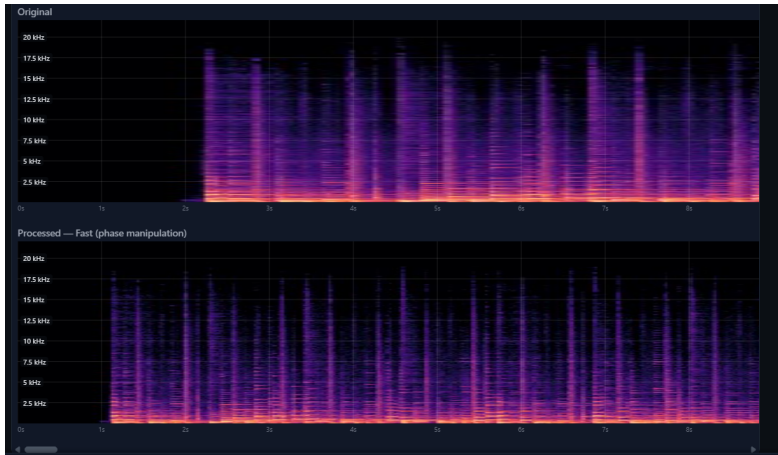
A short ~ 20 ms refractory window stops one long attack from re-triggering every frame it spans.

Resampling: Why Not Just Interpolate Linearly?

- Linear interpolation has **no anti-aliasing** — compressing a signal folds high frequencies back into the audible range as metallic, comb-like artifacts.
- Lanczos (windowed-sinc) resampling has a built-in low-pass cutoff when compressing, so it filters as it resamples.
- Trade-off: more computation per sample (wider kernel) for a clean result.

“Linear resampling aliases; Lanczos filters as it resamples.”

Result: Speed-Only Change (2×, Fast mode)



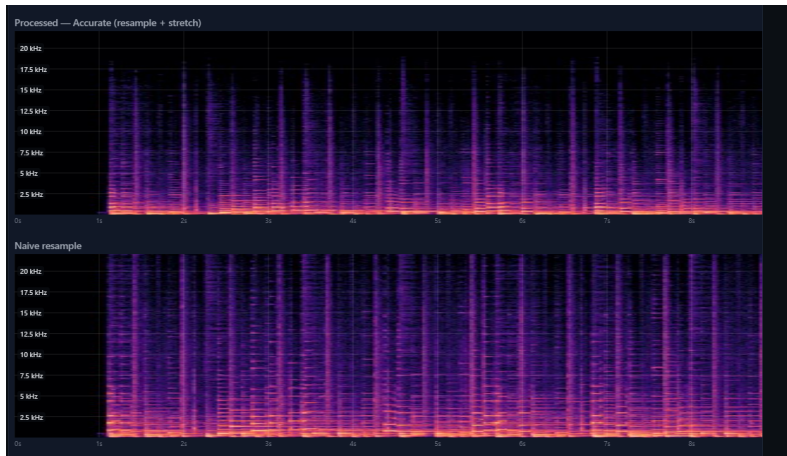
Semitone shift = 0. Notice the harmonic bands stay at the same height while the clip compresses in time — speed changed, pitch did not.

Result: Pitch-Only Change (−9 semitones, Accurate mode)



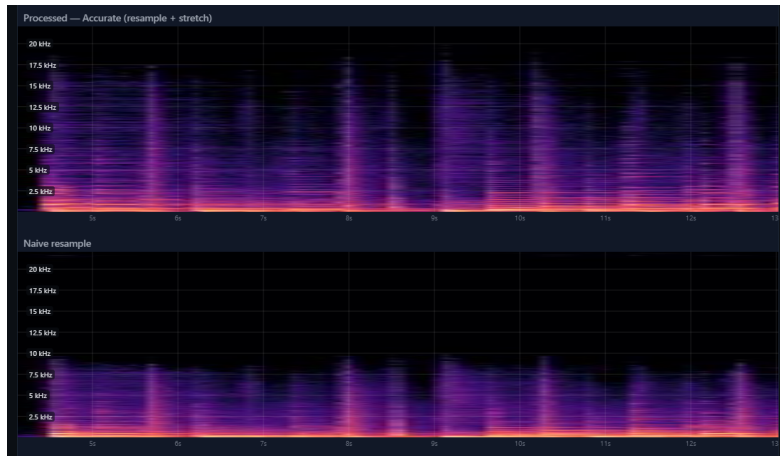
Speed factor = 1.0. Notice the harmonic bands drop to a lower frequency while the timeline is unchanged — pitch changed, duration did not.

Result: Naive vs. Accurate — $2\times$ Speed-Up



Same target duration, two methods. Naive resample's bands rise with the speed-up; our output's bands stay put.

Result: Naive vs. Accurate — $0.5\times$ Speed (Slow-Down)



Same slow-down, two methods. Naive resample's bands drop as the clip is stretched; our output again keeps them fixed.

Evaluation Checklist

Feature	Implemented
Speed control	✓
Pitch shift	✓
Independent speed/pitch control	✓ (within 0.25–3.0× effective stretch)
Transient preservation (speech)	✓
Naive-resample comparison mode	✓
Formant preservation	×
Stereo support	× (mono only, down-mixed on load)

Stated up front, not hidden: mono-only, no formant correction — both are known scope limits, not missing features we forgot.

- Speed and pitch, unglued — with a transient-aware phase vocoder keeping speech intelligible.
- Two black-box algorithms doing the heavy lifting: **phase vocoder** for time, **Lanczos resampling** for pitch.
- Fast and Accurate modes trade cost for pitch-shift range.

Questions?

Appendix: STFT Overlap-Add / WOLA Normalization

Overlap-adding windowed frames reintroduces the window's amplitude ripple. Dividing by the running sum of *squared* windows at each sample position removes it — for Hanning + 75% overlap this sum is nearly constant, so the correction is small.

Appendix: Phase Accumulation

Replaying analysis phase directly at a new hop breaks phase continuity between frames (heard as warbling). Re-accumulating from the *estimated instantaneous frequency* instead keeps phase continuous across the new hop spacing.

Appendix: Why Hann (Hanning) Window

Tapers to exactly zero at both endpoints, suppressing the spectral leakage a hard rectangular edge would introduce. With 75% overlap its squared-sum is nearly constant — the standard choice for WOLA. Implemented by hand: $w[n] = 0.5(1 - \cos(2\pi n/(N - 1)))$.

Appendix: Why Lanczos (Not a Simpler Kernel)

A plain truncated sinc still has large sidelobes. Lanczos windows the sinc with *another* sinc, giving a sharper frequency-domain rolloff for a similar tap count — that rolloff is what supplies anti-aliasing when compressing.

Appendix: Spectral-Flux Threshold Choice

Threshold = $\mu_F + 2\sigma_F$ over the trailing ~ 250 ms of frames (current frame excluded, so a spike can't inflate its own threshold). Two standard deviations is a common "clearly above noise floor" heuristic — not derived from a formal false-positive budget here.

Appendix: Adaptive Threshold, Not a Fixed Gaussian σ

The mean/ σ above are recomputed every frame from a *sliding window* of recent flux values — the threshold adapts to the clip's current energy level rather than using one fixed number for the whole track.

Appendix: Phase Reset Mechanics

On a flagged transient frame, the synthesis phase is simply set to that frame's own analysis phase, bypassing the accumulation recurrence for one frame only; accumulation resumes normally on the next frame.

Appendix: Computational Complexity

Dominant cost: $O(N \log N)$ FFT/IFFT per frame $\times$ number of frames ($\approx$ signal length / hop size). Smaller hop (higher overlap) means more frames; the Lanczos resampler is chunked (250,000 samples/chunk) to keep memory bounded on long clips.

Appendix: Valid Range & Why It's a Hard Wall

The implementation raises an error outside $0.25\times$ – $3.0\times$ effective stretch — a deliberate refuse-rather-than-degrade choice, since phase-tracking quality falls off in a way that's hard to bound gracefully near the edges.